

Assignment - 6

Linear Regression Using Gradient Descent

Objective

The objective of this lab is to implement linear regression from scratch and understand the complete learning process:

Prediction $\rightarrow$ Error $\rightarrow$ Gradient $\rightarrow$ Parameter Update

Students will implement linear regression using NumPy without using a ready-made linear regression function from a machine-learning library.

Software Requirements

- Python 3.x
- NumPy
- Matplotlib
- Jupyter Notebook / Google Colab / Python IDE

Instructions

- Complete all tasks in a single Jupyter Notebook or Python file.
- Do not use `sklearn.linear_model.LinearRegression` for the main implementation.
- Show important intermediate calculations.
- Include the required plots and observations.
- Submit the notebook/source code along with the generated figures.

Part A: Implement Linear Regression from Scratch

Dataset: Diabetes

1. Load the dataset.
2. Display the number of samples and features.
3. Separate X and y.
4. Split the data into training and testing sets.
5. Standardize the features.
6. Implement Multiple Linear Regression from scratch.
7. Train using Gradient Descent.
8. Plot Training Loss vs Iterations.

9. Predict the test-set target values.
10. Calculate MSE and R^2 .
11. Compare different learning rates.
12. Compare the result with sklearn LinearRegression.

Task 1: Create the Dataset

- (a) Print the values of x and y .
- (b) Print the number of training examples.
- (c) Plot x versus y using Matplotlib.

Task 2: Initialize the Model

Implement a linear regression model:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Initialize:

$$\theta_0 = 0, \quad \theta_1 = 0.$$

Use:

$$\alpha = 0.01$$

as the learning rate and

$$\text{number of iterations} = 1000.$$

Write Python code to initialize these parameters.

Part B: Prediction and Error

Task 3: Implement Prediction

Write a Python function:

```
def predict(X, theta0, theta1):  
    ...
```

The function should calculate

$$\hat{y} = \theta_0 + \theta_1 x.$$

Test your function using:

$$\theta_0 = 0, \quad \theta_1 = 1.$$

Print the predicted values.

Task 4: Calculate the Error

For every training example, calculate

$$e_i = \hat{y}_i - y_i.$$

Using

$$\theta_0 = 0, \quad \theta_1 = 1,$$

print:

- actual y ,
- predicted $\hat{y}$,
- error e .

Task 5: Implement the Cost Function

Implement the Mean Squared Error cost:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2.$$

Write a Python function:

```
def compute_cost(X, y, theta0, theta1):
    ...
```

Calculate the cost using:

$$\theta_0 = 0, \quad \theta_1 = 1.$$

Print the obtained cost.

Part C: Gradient Descent**Task 6: Calculate the Gradients**

For simple linear regression, implement:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

and

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_i.$$

Write Python code to calculate both gradients.

Use:

$$x = \{1, 2, 3, 4\}, \quad y = \{3, 5, 7, 8\},$$

with

$$\theta_0 = 0, \quad \theta_1 = 1, \quad \alpha = 0.1.$$

Print:

- (a) predicted values,
- (b) errors,
- (c) $\frac{\partial J}{\partial \theta_0}$,
- (d) $\frac{\partial J}{\partial \theta_1}$.

Task 7: Perform One Gradient-Descent Update

Update the parameters using

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

and

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}.$$

Print the old and new values of θ_0 and θ_1 .

Show the numerical calculation in the notebook in addition to the Python output.

Part D: Complete Training Algorithm

Task 8: Implement the Training Loop

Write a function:

```
def fit(X, y, lr=0.01, n_iters=1000):  
    ...
```

The function should:

1. Initialize θ_0 and θ_1 .
2. Calculate predictions.
3. Calculate the errors.
4. Calculate both gradients.
5. Update θ_0 and θ_1 .
6. Calculate and store the cost.
7. Repeat the process for the specified number of iterations.

Train the model using:

$$x = \{1, 2, 3, 4, 5\}, \quad y = \{3, 5, 7, 8, 11\}.$$

Use:

$$\alpha = 0.01, \quad \text{iterations} = 1000.$$

Report the final values of:

$$\theta_0, \quad \theta_1.$$

Task 9: Plot the Learning Curve

Store the cost after every iteration.

Plot:

Iteration Number vs. $J(\theta_0, \theta_1)$.

The graph must contain:

- x-axis: Iteration
- y-axis: Cost
- appropriate title

Observation: Explain whether the cost decreases during training and what this indicates about the learning process.

Part E: Regression Line and Prediction

Task 10: Plot the Best-Fit Line

Using the learned values of θ_0 and θ_1 , calculate predictions for the training data.

Create a plot containing:

- the original data points,
- the learned regression line.

Label the axes and provide a suitable title.

Task 11: Predict a New Value

Using the trained model, predict y for:

$$x = 6.$$

Print the predicted value.

Also calculate the predictions for:

$$x = 7, \quad x = 8, \quad x = 10.$$

Present the results in a table with columns:

x	Predicted $\hat{y}$
6	
7	
8	
10	

Part F: Effect of Learning Rate

Task 12: Compare Different Learning Rates

Train the same model using the following learning rates:

$$0.001, \quad 0.01, \quad 0.1.$$

Use the same dataset and the same number of iterations.

For each learning rate, record:

- final θ_0 ,
- final θ_1 ,
- final cost.

Plot the cost curves separately for the three learning rates.

Question: Describe how changing the learning rate affects the convergence of gradient descent.

Part G: Analytical Solution

Task 13: Calculate the Closed-Form Solution

For the dataset

$$x = \{1, 2, 3, 4, 5\}, \quad y = \{3, 5, 7, 8, 11\},$$

calculate the parameters using:

$$\theta_1 = \frac{m \sum xy - (\sum x)(\sum y)}{m \sum x^2 - (\sum x)^2}$$

and

$$\theta_0 = \bar{y} - \theta_1 \bar{x}.$$

Show the calculation of:

- (a) $\sum x$,
- (b) $\sum y$,
- (c) $\sum xy$,
- (d) $\sum x^2$,
- (e) $\bar{x}$,
- (f) $\bar{y}$,
- (g) θ_1 ,
- (h) θ_0 .

Write the final regression equation:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Task 14: Compare Gradient Descent and Analytical Solution

Compare the parameters obtained using:

1. gradient descent,
2. analytical solution.

Create a small table showing θ_0 , θ_1 , and the corresponding cost for both methods.

Briefly explain why the results may not be exactly identical when gradient descent is stopped after a finite number of iterations.

Part H: Multiple Linear Regression**Task 15: Implement Multiple Linear Regression**

Consider the model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2.$$

Create the following dataset:

Area (x_1)	Bedrooms (x_2)	Price (y)
1000	2	50
1200	2	60
1500	3	75
1800	3	90
2000	4	105

Implement multiple linear regression using gradient descent.

Your program should:

- (a) initialize the weights and bias;
- (b) calculate predictions;
- (c) calculate the gradients;
- (d) update all parameters;
- (e) train the model;
- (f) print the final parameters.

Note: Feature scaling may be used if necessary for stable gradient-descent training.

Task 16: Prediction with Multiple Features

After training the model, predict the price for a house with:

$$x_1 = 1600, \quad x_2 = 3.$$

Print the predicted value.